

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №2 по курсу
«Операционные системы»

Группа: М8О-209Б-24

Студент: Буланов М.С.

Преподаватель: Тихонов Ф.А

Оценка: _____

Дата: 4.11.25

Москва, 2025

Постановка задачи

Вариант 18.

Составить программу на языке Си, обрабатывающую данные в многопоточном режиме. При обработке использовать стандартные средства создания потоков операционной системы (Windows/Unix). Ограничение максимального количества потоков, работающих в один момент времени, должно быть задано ключом запуска вашей программы.

Так же необходимо уметь продемонстрировать количество потоков, используемое вашей программой с помощью стандартных средств операционной системы.

Найти образец в строке наивным алгоритмом

Общий метод и алгоритм решения

Использованные системные вызовы:

- `clock_gettime(CLOCK_MONOTONIC, &start)` - получение монотонного времени для точного замера производительности
- `pthread_create()` - создание потоков для параллельного поиска
- `pthread_join()` - ожидание завершения всех потоков
- `pthread_mutex_lock()` / `pthread_mutex_unlock()` - синхронизация доступа к общему массиву результатов

Алгоритм работы программы:

1. Инициализация и распределение данных

- Главный поток считывает параметры: максимальное количество потоков, исходный текст и образец для поиска
- Вычисляется реальное количество потоков как минимум между максимальным количеством потоков и количеством возможных позиций для поиска
- Текст разбивается на перекрывающиеся сегменты для каждого потока (перекрытие = длина образца - 1)

2. Параллельный поиск

- Каждый поток выполняет наивный алгоритм поиска в своем сегменте текста
- Наивный алгоритм последовательно проверяет каждую позицию в сегменте, сравнивая символы образца с соответствующими символами текста

- Найденные позиции сохраняются в локальный массив каждого потока

3. Синхронизация результатов

- После завершения поиска каждый поток добавляет свои результаты в общий массив
- Доступ к общему массиву защищается мьютексом для предотвращения гонки данных
- Главный поток ожидает завершения всех рабочих потоков

4. Вывод результатов

- Программа выводит количество найденных вхождений, их позиции в тексте
- Отображается время выполнения и фактическое количество использованных потоков
- Предоставляется PID процесса для демонстрации потоков средствами операционной системы

Код программы

naïve_search.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <time.h>

// Структура для результатов поиска
typedef struct {
    int *positions;          // массив для найденных позиций
    int count;               // количество найденных
    int capacity;            // вместимость массива
    pthread_mutex_t mutex;   // мьютекс для этого результата
} search_results_t;

// Структура для передачи данных в поток
typedef struct {
    const char *text_chunk;  // часть текста для поиска
    int chunk_length;        // длина этой части
    int global_offset;       // смещение в исходном тексте
    const char *pattern;     // образец для поиска
    int pattern_len;         // длина образца
    search_results_t *results; // указатель на общие результаты
} thread_data_t;

// Инициализация структуры результатов
void init_results(search_results_t *results, int initial_capacity) {
    results->positions = malloc(initial_capacity * sizeof(int));
    if (results->positions == NULL) {
        perror("malloc failed");
        exit(1);
    }
    results->count = 0;
    results->capacity = initial_capacity;
```

```

    pthread_mutex_init(&results->mutex, NULL);
}

// Освобождение ресурсов результатов
void free_results(search_results_t *results) {
    free(results->positions);
    pthread_mutex_destroy(&results->mutex);
}

// Добавление позиции в результаты (с синхронизацией)
void add_position(search_results_t *results, int position) {
    pthread_mutex_lock(&results->mutex);

    // Если массив заполнен, увеличиваем его
    if (results->count >= results->capacity) {
        results->capacity *= 2;
        int *new_positions = realloc(results->positions,
                                      results->capacity * sizeof(int));
        if (new_positions == NULL) {
            perror("realloc failed");
            pthread_mutex_unlock(&results->mutex);
            return;
        }
        results->positions = new_positions;
    }

    results->positions[results->count++] = position;
    pthread_mutex_unlock(&results->mutex);
}

// Функция потока для поиска (наивный алгоритм)
void* search_thread(void *arg) {
    thread_data_t *data = (thread_data_t*)arg;

    // Наивный алгоритм поиска подстроки
    for (int i = 0; i <= data->chunk_length - data->pattern_len; i++) {
        int j;
        for (j = 0; j < data->pattern_len; j++) {
            if (data->text_chunk[i + j] != data->pattern[j]) {
                break;
            }
        }

        // Если весь образец совпал
        if (j == data->pattern_len) {
            int global_pos = data->global_offset + i;
            add_position(data->results, global_pos);
        }
    }

    return NULL;
}

// Главная функция многопоточного поиска
void parallel_string_search(const char *text, const char *pattern, int num_threads) {
    size_t text_len = strlen(text);
    int pattern_len = strlen(pattern);

    if (pattern_len == 0) {
        printf("Pattern is empty\n");
        return;
    }
}

```

```

if (text_len < pattern_len) {
    printf("Text is shorter than pattern\n");
    return;
}

// Инициализация результатов
search_results_t results;
init_results(&results, 100);

// Выделяем память для массивов потоков и данных
pthread_t *threads = malloc(num_threads * sizeof(pthread_t));
thread_data_t *thread_data = malloc(num_threads * sizeof(thread_data_t));

if (threads == NULL || thread_data == NULL) {
    perror("malloc failed");
    free_results(&results);
    return;
}

// Разделяем текст на части с перекрытием
size_t chunk_size = text_len / num_threads;

printf("Starting search with %d threads\n", num_threads);
printf("Text length: %zu, Pattern: '%s' (%d chars)\n",
        text_len, pattern, pattern_len);
printf("Chunk size: %zu bytes\n", chunk_size);

// Создаем потоки
for (int i = 0; i < num_threads; i++) {
    // Определяем границы блока с перекрытием
    size_t start = i * chunk_size;
    size_t end;

    if (i == num_threads - 1) {
        // Последний поток получает до конца текста
        end = text_len;
    } else {
        // Обычный блок + перекрытие для поиска на границах
        end = start + chunk_size + pattern_len - 1;
        if (end > text_len) end = text_len;
    }

    thread_data[i].text_chunk = text + start;
    thread_data[i].chunk_length = end - start;
    thread_data[i].global_offset = start;
    thread_data[i].pattern = pattern;
    thread_data[i].pattern_len = pattern_len;
    thread_data[i].results = &results;

    if (pthread_create(&threads[i], NULL, search_thread, &thread_data[i]) != 0) {
        fprintf(stderr, "Failed to create thread %d\n", i);
    }
}

// Ожидаем завершения всех потоков
for (int i = 0; i < num_threads; i++) {
    pthread_join(threads[i], NULL);
}

// Выводим результаты
printf("\nSearch completed. Found %d occurrences:\n", results.count);
for (int i = 0; i < results.count; i++) {
    printf("Position %d\n", results.positions[i]);
}

```

```

    }

    // Освобождаем ресурсы
    free(threads);
    free(thread_data);
    free_results(&results);
}

// Функция для измерения времени
double get_time() {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[]) {
    if (argc != 5) {
        printf("Usage: %s -t <num_threads> <pattern> <text>\n", argv[0]);
        printf("Example: %s -t 4 \"hello\" \"hello world hello there\"\n", argv[0]);
        return 1;
    }

    // Парсинг аргументов
    if (strcmp(argv[1], "-t") != 0) {
        printf("Error: expected -t flag\n");
        return 1;
    }

    int num_threads = atoi(argv[2]);
    const char *pattern = argv[3];
    const char *text = argv[4];

    if (num_threads <= 0) {
        printf("Error: number of threads must be positive\n");
        return 1;
    }

    //printf("Input text: '%s'\n", text);
    //printf("Search pattern: '%s'\n", pattern);
    printf("Number of threads: %d\n", num_threads);

    // Замер времени выполнения
    double start_time = get_time();

    // Выполняется многопоточный поиск
    parallel_string_search(text, pattern, num_threads);

    double end_time = get_time();
    printf("\nExecution time: %.6f seconds\n", end_time - start_time);

    return 0;
}

```

test_threads.sh проверка работы кода

```
#!/bin/bash
```

```
echo "Поиск слова 'Тро' в тексте про Троию"
echo "=====
```

```
ТЕХТ="Двери пред ними разверзла прелестная ликом Феано, \
Дщерь Киссея, жена Антенора, смирителя коней, \
Трои мужами избранная жрица Афины богини.\
```

[illegible]

[illegible]

Search pattern: 'Tpo'

Number of threads: 1

Starting search with 1 threads

Text length: 2259, Pattern: 'Tpo' (6 chars)

Chunk size: 2259 bytes

Search completed. Found 9 occurrences:

Position 175

Position 426

Position 677

Position 928

Position 1179

Position 1430

Position 1681

Position 1932

Position 2183

Execution time: 0.000291 seconds

--- 2 потоков ---

Input text: 'Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои мужами избранная жрица Афины богини.Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои

[illegible]

Search pattern: 'Tpo'

Number of threads: 2

Starting search with 2 threads

Text length: 2259, Pattern: 'Tpo' (6 chars)

Chunk size: 1129 bytes

Search completed. Found 9 occurrences:

Position 175

Position 426

Position 677

Position 928

Position 1179

Position 1430

Position 1681

Position 1932

Position 2183

Execution time: 0.000574 seconds

--- 3 ПОТОКОВ ---

Input text: 'Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои мужами избранная жрица Афины богини. Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои

[illegible]

Search pattern: 'Tpo'

Number of threads: 3

Starting search with 3 threads

Text length: 2259, Pattern: 'Tpo' (6 chars)

Chunk size: 753 bytes

Search completed. Found 9 occurrences:

Position 175

Position 426

Position 677

Position 928

Position 1179

Position 1430

Position 1681

Position 1932

Position 2183

Execution time: 0.001006 seconds

--- 4 ПОТОКОВ ---

Input text: 'Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои мужами избранная жрица Афины богини. Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои

[illegible]

Search pattern: 'Tpo'

Number of threads: 4

Starting search with 4 threads

Text length: 2259, Pattern: 'Tpo' (6 chars)

Chunk size: 564 bytes

Search completed. Found 9 occurrences:

Position 175

Position 426

Position 677

Position 928

Position 1179

Position 1430

Position 1681

Position 1932

Position 2183

Execution time: 0.000988 seconds

--- 5 потоков ---

Input text: 'Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои мужами избранная жрица Афины богини.Двери пред ними разверзла прелестная ликом Феано, Дщерь Киссея, жена Антенора, смирителя коней, Трои

[illegible]

Search pattern: 'Tpo'

Number of threads: 5

Starting search with 5 threads

Text length: 2259, Pattern: 'Tpo' (6 chars)

Chunk size: 451 bytes

Search completed. Found 9 occurrences:

Position 175

Position 426

Position 677

Position 928

Position 1179

Position 1430

Position 1681

Position 1932

Position 2183

Execution time: 0.000603 seconds

STRACE:

Введена команда

```
root@c1caffa1e2c3:/workspaces/Operating_Systems/lab2/build# strace -f ./naive_search -t 4 "ll"
"allaallallllallllllallllalll"
```

```
execve("./naive_search", [ "./naive_search", "-t", "4", "ll", "allaallallllallllllallllalll"], 0x7ffc69a9be18
/* 28 vars */) = 0
```

```
brk(NULL) = 0x565184552000
```

```
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffd1676e280) = -1 EINVAL (Invalid argument)
```

```
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f2650c66000
```

```
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
```

```
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=12460, ...}, AT_EMPTY_PATH) = 0
```

```
mmap(NULL, 12460, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f2650c62000
```

```
close(3) = 0
```

```
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
```

```
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0P\237\2\0\0\0\0\0"..., 832) =
832
```

```
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
```

```
pread64(3, "\4\0\0\0\0\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0\0"..., 48,
848) = 48
```

```
pread64(3,
"\4\0\0\0\24\0\0\0\3\0\0\0GNU\00{\f\225\=\201\327\312\301P\32$\230\266\235"...,
68, 896) = 68
```

```
newfstatat(3, "", {st_mode=S_IFREG|0755, st_size=2220400, ...}, AT_EMPTY_PATH) = 0
```

```
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0@\0\0\0\0\0\0\0"..., 784,
64) = 784
```

```
mmap(NULL, 2264656, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x7f2650a39000
```

```
mprotect(0x7f2650a61000, 2023424, PROT_NONE) = 0
```

```
mmap(0x7f2650a61000, 1658880, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x28000) = 0x7f2650a61000
```

```
mmap(0x7f2650bf6000, 360448, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1bd000) = 0x7f2650bf6000
```

```
mmap(0x7f2650c4f000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x215000) = 0x7f2650c4f000
```

```

mmap(0x7f2650c55000, 52816, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f2650c55000

close(3) = 0

mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) =
0x7f2650a36000

arch_prctl(ARCH_SET_FS, 0x7f2650a36740) = 0

set_tid_address(0x7f2650a36a10) = 26724

set_robust_list(0x7f2650a36a20, 24) = 0

rseq(0x7f2650a370e0, 0x20, 0, 0x53053053) = 0

mprotect(0x7f2650c4f000, 16384, PROT_READ) = 0

mprotect(0x565183e39000, 4096, PROT_READ) = 0

mprotect(0x7f2650ca0000, 8192, PROT_READ) = 0

prlimit64(0, RLIMIT_STACK, NULL, {rlim_cur=8192*1024, rlim_max=RLIM64_INFINITY}) = 0

munmap(0x7f2650c62000, 12460) = 0

newfstatat(1, "", {st_mode=S_IFCHR|0620, st_rdev=makedev(0x88, 0x3), ...}, AT_EMPTY_PATH) = 0

getrandom("\xdc\x5e\x25\xe7\xbb\xa5\x07\x01", 8, GRND_NONBLOCK) = 8

brk(NULL) = 0x565184552000

brk(0x565184573000) = 0x565184573000

write(1, "Number of threads: 4\n", 21Number of threads: 4
) = 21

write(1, "Starting search with 4 threads\n", 31Starting search with 4 threads
) = 31

write(1, "Text length: 29, Pattern: 'll' ("..., 41Text length: 29, Pattern: 'll' (2 chars)
) = 41

write(1, "Chunk size: 7 bytes\n", 20Chunk size: 7 bytes
) = 20

rt_sigaction(SIGRT_1, {sa_handler=0x7f2650aca870, sa_mask=[],
sa_flags=SA_RESTORER|SA_ONSTACK|SA_RESTART|SA_SIGINFO, sa_restorer=0x7f2650a7b520},
NULL, 8) = 0

rt_sigprocmask(SIG_UNBLOCK, [RTMIN RT_1], NULL, 8) = 0

mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK, -1, 0) =
0x7f2650235000

mprotect(0x7f2650236000, 8388608, PROT_READ|PROT_WRITE) = 0

rt_sigprocmask(SIG_BLOCK, ~[], [], 8) = 0

```

```
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY
SVSEM|CLONE_SETTTL|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7f2650a35910, parent_tid=0x7f2650a35910, exit_signal=0, stack=0x7f2650235000,
stack_size=0x7fff00, tls=0x7f2650a35640}, 88) = -1 ENOSYS (Function not implemented)
```

```
clone(child_stack=0x7f2650a34ef0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSE
M|CLONE_SETTTL|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTIDstrace: Process 26725
attached
```

```
, parent_tid=[26725], tls=0x7f2650a35640, child_tidptr=0x7f2650a35910) = 26725
```

```
[pid 26725] rseq(0x7f2650a35fe0, 0x20, 0, 0x53053053 <unfinished ...>
```

```
[pid 26724] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>
```

```
[pid 26725] <... rseq resumed>      = 0
```

```
[pid 26724] <... rt_sigprocmask resumed>NULL, 8) = 0
```

```
[pid 26725] set_robust_list(0x7f2650a35920, 24 <unfinished ...>
```

```
[pid 26724] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0 <unfinished ...>
```

```
[pid 26725] <... set_robust_list resumed>) = 0
```

```
[pid 26724] <... mmap resumed>      = 0x7f264fa34000
```

```
[pid 26725] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>
```

```
[pid 26724] mprotect(0x7f264fa35000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>
```

```
[pid 26725] <... rt_sigprocmask resumed>NULL, 8) = 0
```

```
[pid 26724] <... mprotect resumed>) = 0
```

```
[pid 26725] rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>
```

```
[pid 26724] rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>
```

```
[pid 26725] <... rt_sigprocmask resumed>NULL, 8) = 0
```

```
[pid 26724] <... rt_sigprocmask resumed>[], 8) = 0
```

```
[pid 26725] madvise(0x7f2650235000, 8368128, MADV_DONTNEED <unfinished ...>
```

```
[pid 26724]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY
SVSEM|CLONE_SETTTL|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7f2650234910, parent_tid=0x7f2650234910, exit_signal=0, stack=0x7f264fa34000,
stack_size=0x7fff00, tls=0x7f2650234640} <unfinished ...>
```

```
[pid 26725] <... madvise resumed>    = 0
```

```
[pid 26724] <... clone3 resumed>, 88) = -1 ENOSYS (Function not implemented)
```

```
[pid 26725] exit(0 <unfinished ...>
```



```

[pid 26724] clone(child_stack=0x7f2650233ef0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSE
M|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID <unfinished ...>

[pid 26725] <... exit resumed>)      = ?

strace: Process 26726 attached

[pid 26725] +++ exited with 0 +++

[pid 26724] <... clone resumed>, parent_tid=[26726], tls=0x7f2650234640,
child_tidptr=0x7f2650234910) = 26726

[pid 26726] rseq(0x7f2650234fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 26724] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 26726] <... rseq resumed>)      = 0

[pid 26724] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26726] set_robust_list(0x7f2650234920, 24 <unfinished ...>

[pid 26724] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0 <unfinished ...>

[pid 26726] <... set_robust_list resumed>) = 0

[pid 26724] <... mmap resumed>)      = 0x7f264f233000

[pid 26726] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 26724] mprotect(0x7f264f234000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 26726] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26724] <... mprotect resumed>)  = 0

[pid 26726] rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>

[pid 26724] rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>

[pid 26726] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26724] <... rt_sigprocmask resumed>[], 8) = 0

[pid 26726] madvise(0x7f264fa34000, 8368128, MADV_DONTNEED <unfinished ...>

[pid 26724]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY
SVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7f264fa33910, parent_tid=0x7f264fa33910, exit_signal=0, stack=0x7f264f233000,
stack_size=0x7fff00, tls=0x7f264fa33640} <unfinished ...>

[pid 26726] <... madvise resumed>)    = 0

[pid 26724] <... clone3 resumed>, 88) = -1 ENOSYS (Function not implemented)

[pid 26726] exit(0 <unfinished ...>

```

[pid 26724] clone(child_stack=0x7f264fa32ef0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSE
M|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID <unfinished ...>

[pid 26726] <... exit resumed> = ?

[pid 26726] +++ exited with 0 +++

strace: Process 26727 attached

[pid 26724] <... clone resumed>, parent_tid=[26727], tls=0x7f264fa33640,
child_tidptr=0x7f264fa33910) = 26727

[pid 26727] rseq(0x7f264fa33fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 26724] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 26727] <... rseq resumed> = 0

[pid 26724] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26727] set_robust_list(0x7f264fa33920, 24 <unfinished ...>

[pid 26724] mmap(NULL, 8392704, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_STACK,
-1, 0 <unfinished ...>

[pid 26727] <... set_robust_list resumed> = 0

[pid 26724] <... mmap resumed> = 0x7f264ea32000

[pid 26727] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 26724] mprotect(0x7f264ea33000, 8388608, PROT_READ|PROT_WRITE <unfinished ...>

[pid 26727] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26724] <... mprotect resumed> = 0

[pid 26727] rt_sigprocmask(SIG_BLOCK, ~[RT_1], <unfinished ...>

[pid 26724] rt_sigprocmask(SIG_BLOCK, ~[], <unfinished ...>

[pid 26727] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26724] <... rt_sigprocmask resumed>[], 8) = 0

[pid 26727] madvise(0x7f264f233000, 8368128, MADV_DONTNEED <unfinished ...>

[pid 26724]
clone3({flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SY
SVSEM|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEARTID,
child_tid=0x7f264f232910, parent_tid=0x7f264f232910, exit_signal=0, stack=0x7f264ea32000,
stack_size=0x7fff00, tls=0x7f264f232640} <unfinished ...>

[pid 26727] <... madvise resumed> = 0

[pid 26724] <... clone3 resumed>, 88) = -1 ENOSYS (Function not implemented)

[pid 26727] exit(0 <unfinished ...>

```

[pid 26724] clone(child_stack=0x7f264f231ef0,
flags=CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND|CLONE_THREAD|CLONE_SYSVSE
M|CLONE_SETTLS|CLONE_PARENT_SETTID|CLONE_CHILD_CLEAR_TID <unfinished ...>

[pid 26727] <... exit resumed>      = ?

strace: Process 26728 attached

[pid 26727] +++ exited with 0 +++

[pid 26724] <... clone resumed>, parent_tid=[26728], tls=0x7f264f232640,
child_tidptr=0x7f264f232910) = 26728

[pid 26728] rseq(0x7f264f232fe0, 0x20, 0, 0x53053053 <unfinished ...>

[pid 26724] rt_sigprocmask(SIG_SETMASK, [], <unfinished ...>

[pid 26728] <... rseq resumed>      = 0

[pid 26724] <... rt_sigprocmask resumed>NULL, 8) = 0

[pid 26728] set_robust_list(0x7f264f232920, 24 <unfinished ...>

[pid 26724] futex(0x7f264f232910, FUTEX_WAIT_BITSET|FUTEX_CLOCK_REALTIME, 26728,
NULL, FUTEX_BITSET_MATCH_ANY <unfinished ...>

[pid 26728] <... set_robust_list resumed>) = 0

[pid 26728] rt_sigprocmask(SIG_SETMASK, [], NULL, 8) = 0

[pid 26728] rt_sigprocmask(SIG_BLOCK, ~[RT_1], NULL, 8) = 0

[pid 26728] madvise(0x7f264ea32000, 8368128, MADV_DONTNEED) = 0

[pid 26728] exit(0)                = ?

[pid 26724] <... futex resumed>     = 0

[pid 26728] +++ exited with 0 +++

write(1, "\n", 1
)
= 1

write(1, "Search completed. Found 16 occur"..., 40Search completed. Found 16 occurrences:
) = 40

write(1, "Position 1\n", 11Position 1
)
= 11

write(1, "Position 5\n", 11Position 5
)
= 11

write(1, "Position 6\n", 11Position 6
)
= 11

write(1, "Position 9\n", 11Position 9
)
= 11

```

```

write(1, "Position 10\n", 12Position 10
)      = 12
write(1, "Position 11\n", 12Position 11
)      = 12
write(1, "Position 14\n", 12Position 14
)      = 12
write(1, "Position 15\n", 12Position 15
)      = 12
write(1, "Position 16\n", 12Position 16
)      = 12
write(1, "Position 17\n", 12Position 17
)      = 12
write(1, "Position 18\n", 12Position 18
)      = 12
write(1, "Position 21\n", 12Position 21
)      = 12
write(1, "Position 22\n", 12Position 22
)      = 12
write(1, "Position 23\n", 12Position 23
)      = 12
write(1, "Position 26\n", 12Position 26
)      = 12
write(1, "Position 27\n", 12Position 27
)      = 12
write(1, "\n", 1
)      = 1
write(1, "Execution time: 0.013274 seconds"..., 33Execution time: 0.013274 seconds
) = 33
exit_group(0)      = ?
+++ exited with 0 +++

```

Скрипт bash для получения данных для таблиц test_threads.sh в папке лабораторной работы

Вывод

Таблица с расчетами для текста длиной 502 символа:

Таблица производительности:

Потоки	Время (сек)	Ускорение	Эффективность (%)
1	0.000213	1.00x	100.0%
2	0.000275	0.77x	38.7%
3	0.000461	0.46x	15.4%
4	0.000511	0.42x	10.4%
5	0.001078	0.20x	4.0%

Таблица с расчетами для текста длиной 7803 символа:

Потоки	Время (сек)	Ускорение	Эффективность (%)
1	0.000598	1.00x	100.0%
2	0.001106	0.54x	27.0%
3	0.000675	0.89x	29.5%
4	0.000655	0.91x	22.8%
5	0.001000	0.60x	12.0%

Для хорошей эффективности нужны большие тексты на много тысяч символов, с увеличением количества символов растет эффективность. Примечательно что с увеличением размера текста сильно растет эффективность для большого количества потоков